

AI Diet & Fitness Agent

Natural Language → Tool Calling → Structured State → Nutrition Intelligence

Project Status: Phase 1 — Core Agent System

Stack: Python · LangChain · LangGraph · Groq · Tavily · Pydantic

Documentation

Prepared by

Amirtha Ganesh R.

August 17, 2026

Contents

1	Project Idea	2
1.1	Overall Concept	2
2	Technologies Used	2
2.1	Model History (Groq)	3
3	Project Structure	3
4	Data Models	3
4.1	UserProfile	3
4.2	FoodEntry	3
4.3	WorkoutEntry	4
4.4	NutritionInfo	4
5	Agent State	4
6	Tools	4
6.1	Food Logging Tool — <code>add_food</code>	4
6.2	Workout Logging Tool — <code>add_workout</code>	5
6.3	Nutrition Retrieval	5
6.4	Nutrition Extraction with Groq	5
6.5	Structured LLM Output	5
6.6	Daily Summary Tool — <code>daily_summary</code>	5
7	The Agent	6
8	The Important Architecture Problem We Solved	6
8.1	The Fix	6
9	Final Agent Flow	7
10	End-to-End Example	7
11	What Has Been Successfully Tested	7
12	Problems We Encountered & Solved	8
13	Current Status	8
13.1	Completed	9
13.2	Remaining Work	9
14	Final Planned Architecture	10
15	What This Project Demonstrates	10

1 Project Idea

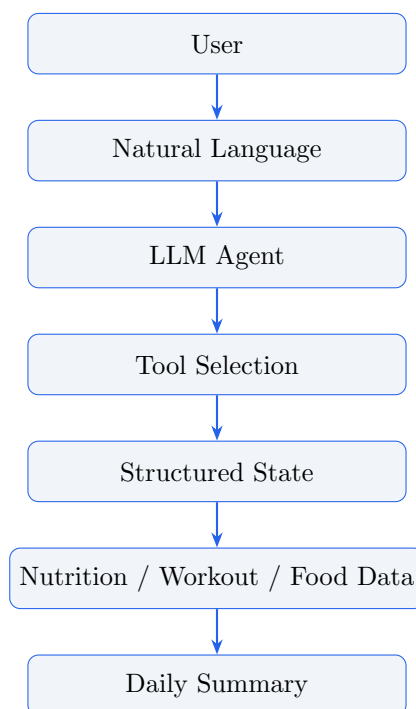
The goal is to build an AI-powered Diet & Fitness Agent that allows a user to interact naturally instead of manually entering structured data.

Example prompt:

"I ate 2 eggs for breakfast and then did a 45 minute chest workout. Give me today's summary."

The agent should understand the request, determine what actions are required, call the appropriate tools, update the user's state, retrieve nutrition information, and generate a meaningful summary.

1.1 Overall Concept



2 Technologies Used

Python

Core programming language.

Pydantic

Used for creating structured and validated data models.

LangChain

Used for LLM integration, tool creation, structured output, and agent creation.

LangGraph

Used underneath the agent architecture for managing agent state and tool execution.

Groq

Used as the LLM provider.

Tavily

Used to search the web for nutrition information.

dotenv

Used to load API keys from `.env`.

2.1 Model History (Groq)

- **Current model:** openai/gpt-oss-120b
- **Previously tested:** llama-3.3-70b-versatile, qwen/qwen3.6-27b

The original Llama model became unavailable, and Qwen produced excessive reasoning output for this simple tool-calling workflow, so openai/gpt-oss-120b is currently being used.

3 Project Structure

```
diet_agent/  
|  
|-- agent/  
|   '-- diet_agent.py  
|  
|-- tools/  
|   |-- add_food.py  
|   |-- add_workout.py  
|   |-- nutrition.py  
|   '-- summary.py  
|  
|-- models.py  
|-- state.py  
|  
'-- main.py
```

The exact files may evolve as the project grows, but this is the current logical structure.

4 Data Models

The project uses Pydantic models instead of passing unstructured dictionaries everywhere.

4.1 UserProfile

Stores the user's basic information: `name`, `age`, `sex`, `height`, `weight`, `goal`, `workout_type`, `diet_type`.

```
Name: Ganesh  
Age: 24  
Sex: Male  
Height: 178 cm  
Weight: 74 kg  
Goal: General Fitness  
Workout: Gym  
Diet: Non-vegetarian
```

4.2 FoodEntry

Represents a food consumed by the user: `food_name`, `quantity`, `unit`, `time`.

```
Food: Eggs  
Quantity: 2  
Unit: pieces  
Time: now
```

4.3 WorkoutEntry

Represents a workout: `focus`, `duration`, `unit`, `time`.

```
Focus: Chest
Duration: 45
Unit: minutes
Time: now
```

A sensible default for `time` was added so the agent doesn't get stuck when the user doesn't explicitly provide it.

4.4 NutritionInfo

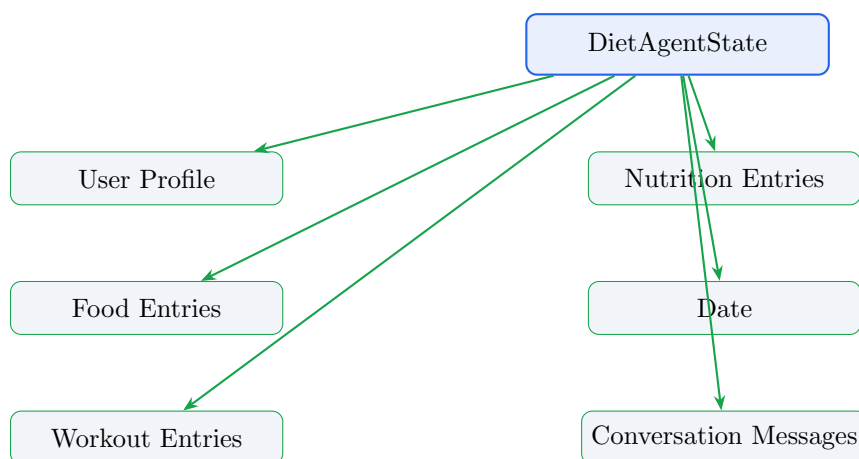
Nutrition information is represented in structured form: `food_name`, `calories`, `protein_g`, `carbs_g`, `fat_g`.

```
food_name: egg
calories: 77.5
protein_g: 6.3
carbs_g: 0.56
fat_g: 5.3
```

This allows the agent to work with actual numerical data instead of raw web-search text.

5 Agent State

One of the most important parts built was `DietAgentState`. The state contains: `messages`, `user`, `food_entries`, `workout_entries`, `nutrition_entries`, `date`.



This gives the agent a structured representation of the user's current day.

6 Tools

6.1 Food Logging Tool — `add_food`

```
FoodEntry -> add_food -> state["food_entries"]
```

The key architectural change was using `runtime.state` instead of asking the LLM to construct the entire state. This prevents the LLM from inventing information.

6.2 Workout Logging Tool — add_workout

```
WorkoutEntry -> add_workout -> state["workout_entries"]
```

Example: *"I did a 45 minute chest workout."*

The agent identifies `focus = Chest`, `duration = 45`, `unit = minutes` and calls `add_workout`, which stores the workout in the real agent state.

6.3 Nutrition Retrieval

Instead of maintaining a massive local database containing every food, the project uses Tavily.

```
Food -> Tavily Search -> Nutrition webpages -> Raw nutrition text
```

For example, `Egg` causes Tavily to search for: *"egg nutrition calories protein carbohydrates fat"*.

6.4 Nutrition Extraction with Groq

Raw web results are messy and can include calories, fat, carbohydrates, protein, vitamins, minerals, serving sizes, percent daily values, and source information — far more than is needed.

```
Tavily -> Raw Web Results -> Groq -> Structured Output -> NutritionInfo
```

The LLM is instructed to extract only: food name, calories, protein, carbohydrates, and fat, returned as the Pydantic `NutritionInfo` model.

6.5 Structured LLM Output

```
llm.with_structured_output(NutritionInfo)
```

Instead of a free-text answer such as *"Eggs contain around 77 calories..."*, the model produces structured data:

```
NutritionInfo(  
    food_name="egg",  
    calories=77.5,  
    protein_g=6.3,  
    carbs_g=0.56,  
    fat_g=5.3  
)
```

This makes the output directly usable by Python code.

6.6 Daily Summary Tool — daily_summary

It reads the current state and produces the date, food consumed, nutrition totals, and workouts.

```
Date: 2026-08-17  
  
Food consumed:  
- eggs: 2 pieces at now  
  
Nutrition:  
- Calories: 155.00 kcal  
- Protein: 12.60 g  
- Carbs: 1.12 g
```

```
- Fat: 10.60 g

Workouts:
- Chest: 45 minutes at now
```

7 The Agent

The tools are connected to the LLM via `create_agent()`. The agent has access to `add_food`, `add_workout`, `extract_nutrition`, and `daily_summary`. The LLM decides which tools are required based on the user's natural-language request.

8 The Important Architecture Problem We Solved

Initially, the LLM was generating the entire state directly, which caused problems: the model started inventing data such as a fictional user named "John", age 30, 70 kg, "Maintain weight", year 2024 — instead of using the actual user's state.

8.1 The Fix

```
REAL APPLICATION STATE -> Agent Runtime -> Tools
```

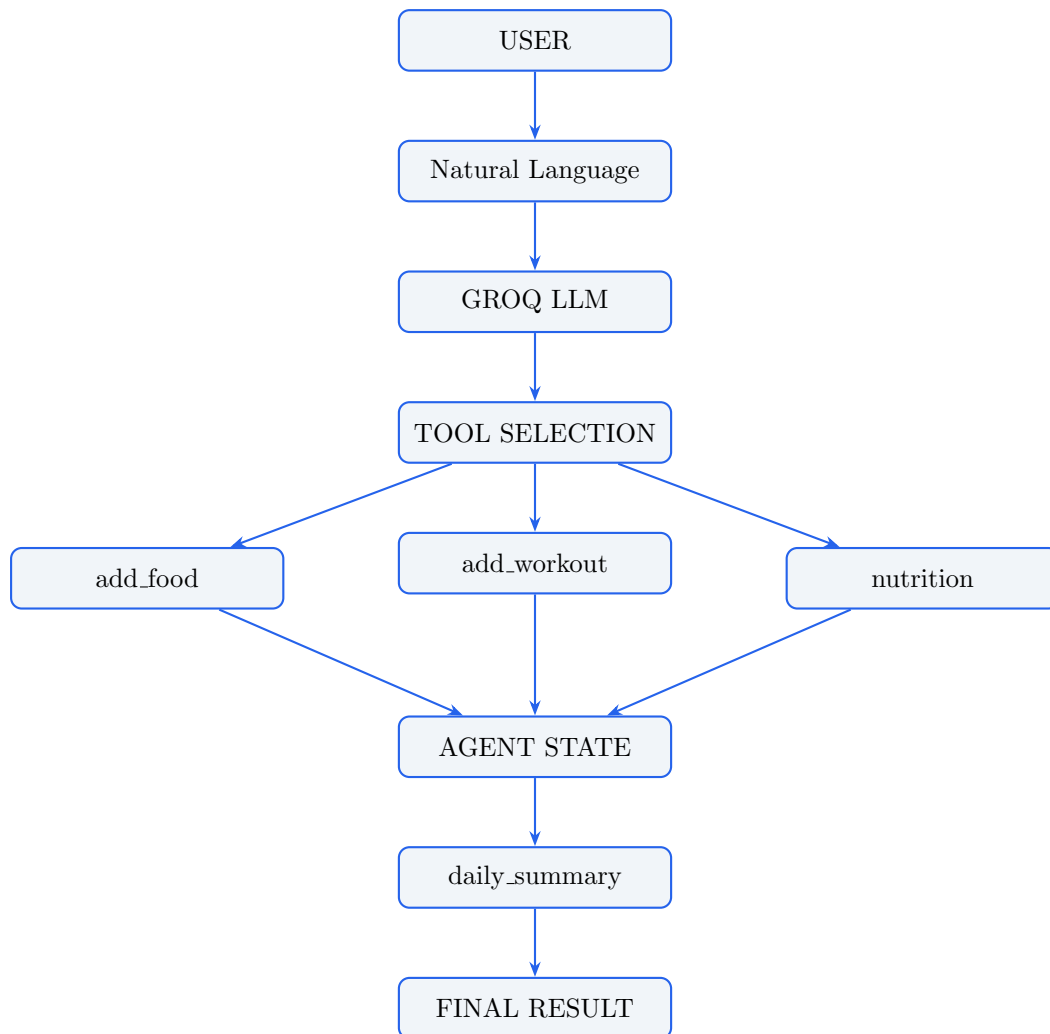
The LLM only supplies the information required for a tool call. For example:

User: *"I did a 45 minute chest workout."*

```
add_workout(
  workout={
    focus="Chest",
    duration=45,
    unit="minutes"
  }
)
```

The tool then accesses the real state through `runtime.state`. This was a major architectural improvement.

9 Final Agent Flow



10 End-to-End Example

Demo input:

"I ate 2 eggs for breakfast. Then I did a 45 minute chest workout. Give me my daily summary."

The agent interprets the request and performs multiple actions:

```
User -> LLM -> add_food -> extract_nutrition -> add_workout -> daily_summary
-> Final response
```

This demonstrates multi-tool agent orchestration rather than a simple chatbot.

11 What Has Been Successfully Tested

- **Food:** *"I ate 2 eggs"* → the agent successfully called `add_food` and stored the food entry.
- **Nutrition:** Tavily successfully retrieved nutrition information; Groq successfully converted the retrieved text into `NutritionInfo`.
- **Daily Summary:** Successfully generated summaries containing food, calories, protein, carbohydrates, fat, and workouts.

- **Workout:** "I did a 45 minute chest workout" → the agent called `add_workout` and returned: "Added Chest workout to today's log."

12 Problems We Encountered & Solved

Problem

1. LangChain tool docstring — `ValueError: Function must have a docstring`
Solved by adding tool descriptions/docstrings.

Problem

2. Incorrect imports — Running `python agent/diet_agent.py` caused `ModuleNotFoundError: No module named 'tools'`
Solved by running the package correctly: `python -m agent.diet_agent`

Problem

3. Missing messages — The LangChain agent expected `messages` in the state.
Solved by adding the messages channel to `DietAgentState`.

Problem

4. Fake state generated by LLM — The LLM initially generated its own user profile.
Solved using `ToolRuntime` and real application state.

Problem

5. Pydantic object vs. dictionary — Runtime state was a dictionary while individual entries remained Pydantic objects.
Solved by correcting access patterns: `runtime.state["food_entries"]`, while entries remain `food.food_name`.

Problem

6. Groq model availability — `llama-3.3-70b-versatile` became unavailable.
Solved by switching models.

Problem

7. Groq rate limit — Hit the Groq daily token limit during development.
Note: This was an API quota issue, not an application architecture issue.

Problem

8. Reasoning-heavy model — `qwen/qwen3.6-27b` generated excessive reasoning for a simple tool call.
Solved by switching to `openai/gpt-oss-120b`, which currently gives clean tool-calling behavior.

13 Current Status

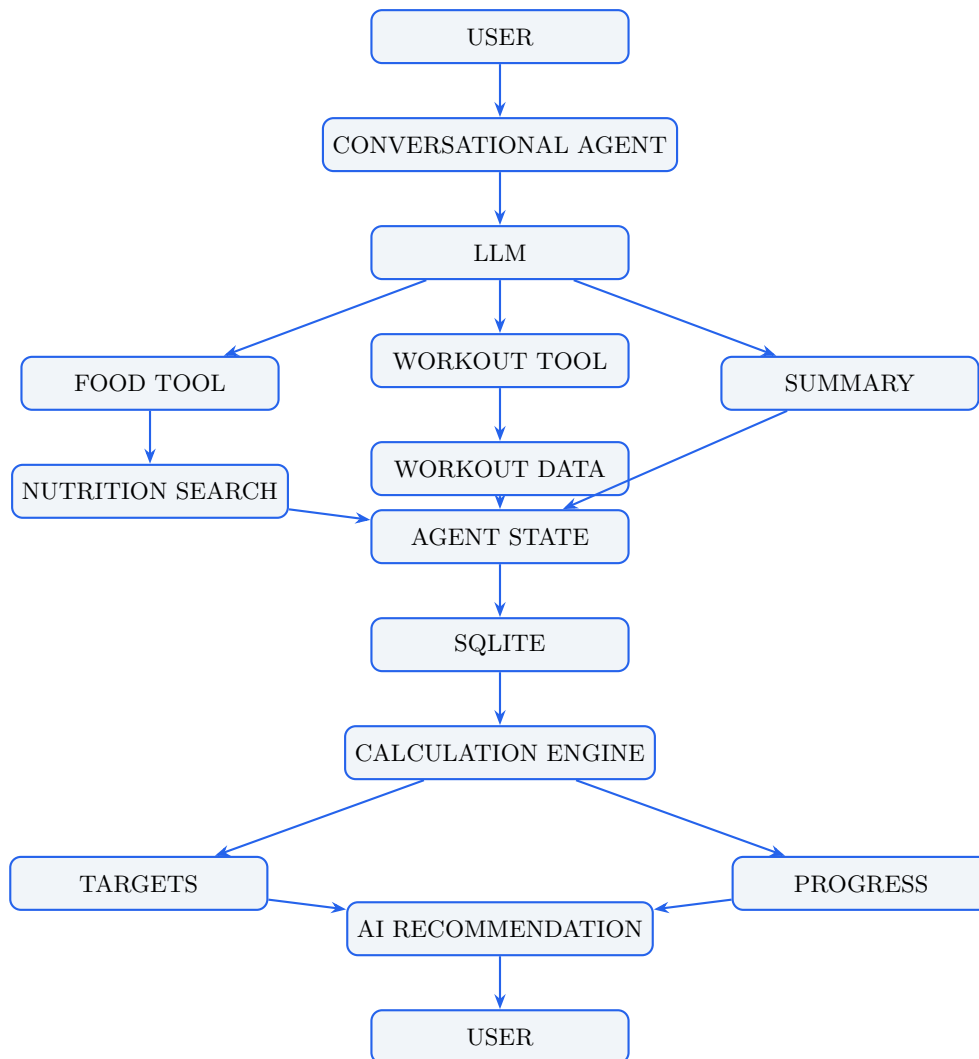
13.1 Completed

- ✓ Project architecture
- ✓ Pydantic models
- ✓ User profile
- ✓ Agent state
- ✓ Food logging
- ✓ Workout logging
- ✓ Tavily nutrition retrieval
- ✓ Groq nutrition extraction
- ✓ Structured nutrition output
- ✓ Daily summary
- ✓ LangChain agent
- ✓ Tool calling
- ✓ Runtime state integration
- ✓ End-to-end testing

13.2 Remaining Work

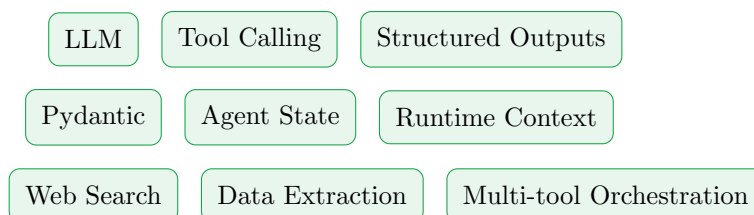
1. **Nutrition quantity handling** — “*I ate 2 eggs*” can currently result in nutrition being retrieved for 1 egg instead of reliably calculating for 2 eggs. This needs to be fixed.
2. **Persistent storage** — State currently exists only during the application run. A SQLite / database layer is needed so data survives after the program stops.
3. **Conversational memory** — Proper persistent conversational/state management is needed to support multi-turn logging and follow-up queries (e.g. “How much protein have I consumed?”).
4. **Nutrition calculations** — BMR, TDEE, daily calorie target, protein target, carbohydrate target, and fat target, based on the user’s profile and goal.
5. **Progress tracking** — Daily target → Consumed → Remaining (e.g. Calories — Target: 2400, Consumed: 1450, Remaining: 950).
6. **AI Recommendations** — Combine user profile, goal, food consumed, nutrition, workout, and remaining targets to generate personalized recommendations via the LLM.
7. **Backend / UI** — FastAPI backend with a Streamlit / web UI once the core system is complete.

14 Final Planned Architecture



15 What This Project Demonstrates

This project isn't simply *"I connected ChatGPT to a nutrition API."* The actual engineering concepts demonstrated are:



The important part is that the LLM is being used as a **decision-making layer**, while deterministic Python tools handle the actual data operations.